- *Blazor Server*—Used to build stateful applications, rendered on the server, that send UI events and page updates over WebSockets to provide the feel of a client-side single-page application, but with the ease of development of a server-rendered application.

All these paradigms are based on the same building blocks of ASP.NET Core, such as the configuration and logging libraries, and then place extra functionality on top. The best paradigm for your application depends on multiple factors, including your API requirements, the details of existing applications you need to interact with, the details of your customers' browsers and operating environment, and scalability and uptime requirements. You don't need to choose only one of these paradigms; ASP.NET Core can combine multiple paradigms within a single application.

1.3 Choosing ASP.NET Core

I hope that now you have a general grasp of what ASP.NET Core is and the type of applications you can build with it. But one question remains: should you use it? Microsoft recommends that all new .NET web development use ASP.NET Core, but switching to or learning a new web stack is a big ask for any developer or company.

If you're new to .NET development and are considering ASP.NET Core, welcome! Microsoft is pushing ASP.NET Core as an attractive option for web development beginners, but taking .NET cross-platform means that it's competing with many other frameworks on their own turf. ASP.NET Core has many selling points compared with other cross-platform web frameworks:

- It's a modern, high-performance, open-source web framework.